

What is Reversing with Trolley ?

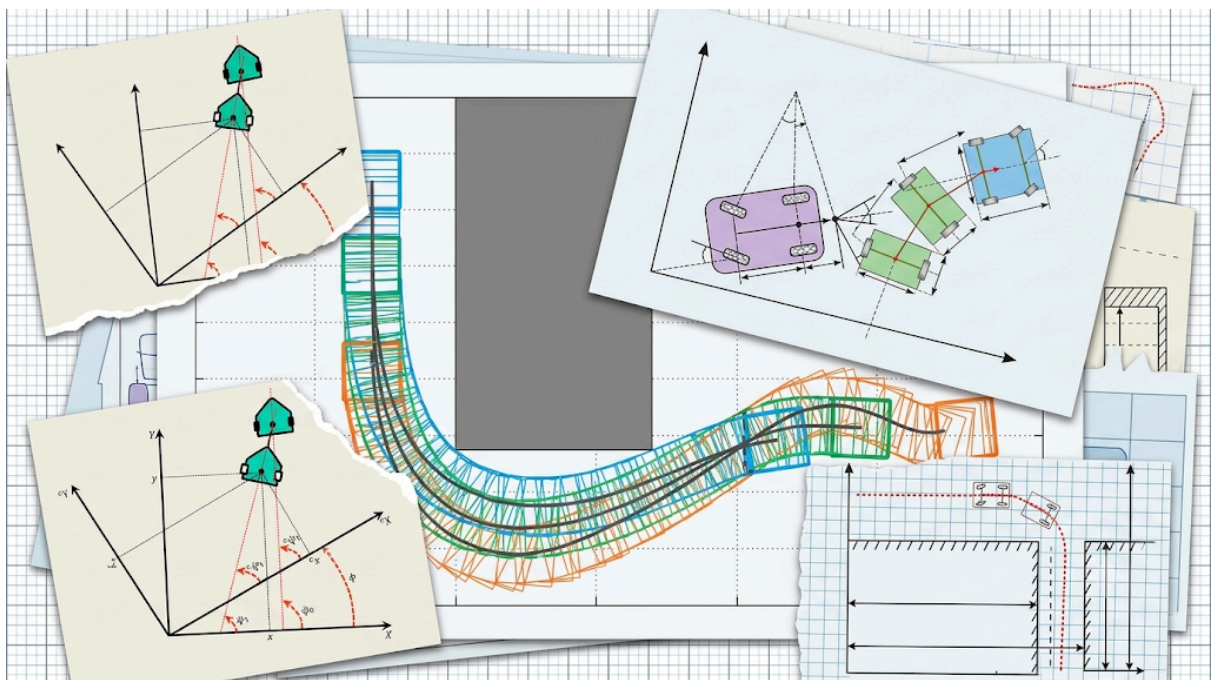
Simply means Reversing the Tractor with Trolley its a hard problem because it an unstable articulated system and its almost like controlling a inverted pendulum.

What was there to start with !

The MPC controller in Frenet Frame with CasADi and Fatrop solver. The states of the MPC were only the Tractor states, not the Trailer, the controller has no idea about where the trailer is and how to control the trailer.

So i started with Augmenting the MPC with Trailer specific states and added the kinematics of the trailer so as to have control over the Trailer as well.

Thumbnail Image :



Here is the Augmented States equation :

$$\mathbf{x} = \begin{bmatrix} s_1 \\ y_1 \\ \bar{\theta}_0 \\ v_0 \\ \phi_0 \\ \bar{\theta}_1 \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} a_0 \\ \omega_\phi \\ \dot{s} \end{bmatrix} \quad \frac{d}{dt} \begin{bmatrix} s_1 \\ y_1 \\ \bar{\theta}_0 \\ v_0 \\ \phi_0 \\ \bar{\theta}_1 \end{bmatrix} = \begin{bmatrix} v_0 \cos \bar{\theta}_0 - \dot{s}(1 - \kappa y_1) \\ v_0 \sin \bar{\theta}_0 - \dot{s} \kappa s_1 \\ \frac{v_0 \tan \phi_0}{L_W} - \kappa \dot{s} \\ a_0 \\ \omega_\phi \\ \frac{v_0 \sin(\bar{\theta}_0 - \bar{\theta}_1)}{L_{W1}} - \frac{L_{H0} \cos(\bar{\theta}_0 - \bar{\theta}_1)}{L_{W1}} \cdot \frac{v_0 \tan \phi_0}{L_W} - \kappa \dot{s} \end{bmatrix}$$

What is the point of having s_1 and y_1 of trailer in State Space ?

The MPC can take it as a regulation constraint and try to make the s_1 and y_1 of trailer as 0 so the trailer follows the path as tractor.

If we remove the variables from State space , its not constrained at all ?

Yes , So I constrained it in the Cost function

$$J = \sum_{k=0}^{N-1} \left(\underbrace{Q_s s_{1,k}^2 + Q_y y_{1,k}^2 + Q_{\theta_0} \bar{\theta}_{0,k}^2}_{\text{Tractor Tracking}} + \underbrace{Q_{y,tr} \left(y_{1,k} - L_1 \sin(\bar{\theta}_{1,k}) \right)^2 + Q_{\theta_1} \bar{\theta}_{1,k}^2}_{\text{Trailer Tracking (6D Projection)}} \right. \\ \left. + \underbrace{R_a a_{0,k}^2 + R_\omega \omega_{\phi,k}^2 + R_s \dot{s}_k^2}_{\text{Control Effort}} + \underbrace{R_{\Delta a} \Delta a_{0,k}^2 + R_{\Delta \omega} \Delta \omega_{\phi,k}^2 + R_{\Delta s} \Delta \dot{s}_k^2}_{\text{Actuator Jerk}} \right. \\ \left. + \underbrace{W_{jack} \left(\frac{\bar{\theta}_{0,k} - \bar{\theta}_{1,k}}{\gamma_{maz}} \right)^{10}}_{\text{Soft Jackknife Barrier}} + \underbrace{P_y y_{1,N}^2 + P_\theta \bar{\theta}_{0,N}^2}_{\text{Terminal Cost}} \right) \longrightarrow \begin{array}{l} y_1 \text{ calculated} \\ \text{geometrically} \\ \text{, not as state} \\ \text{variable} \end{array}$$

Once this is done the next step was to take it to real Bot, But how do we get the Trolley pose. One of my colleague worked in estimating the trolley pose with reflective tapes and Lidar. I wired that algorithm with the controller developed and made it a successful System.

The equation is a draft equation, is not implemented exactly and not much details are provided to preserve NDA

Here is the Video Proof :

https://drive.google.com/file/d/1rZ3Bz4De_voNd6NiShxEsbiK3uJucamC/view?usp=sharing